

PRAKTIKUM SISTEM OPERASI

MODUL 11

MEMORI XINU



Oleh:

Candra Dinata - 2311104061

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK

DIREKTORAT KAMPUS PURWOKERTO

UNIVERSITAS TELKOM

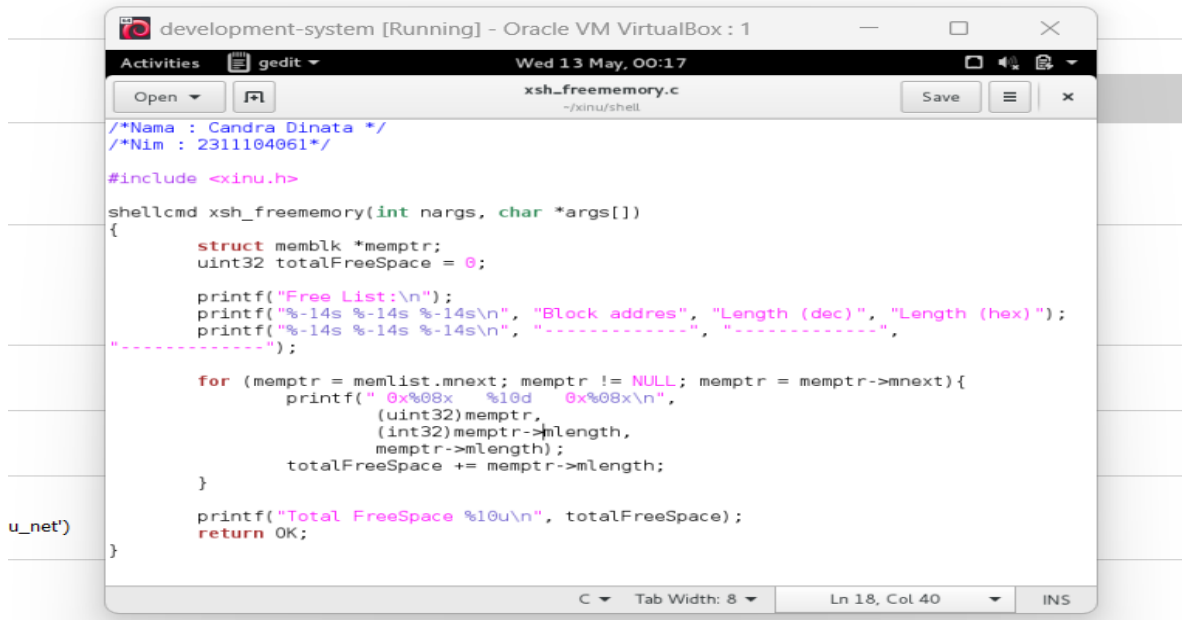
2026

JURNAL

1. Freememory

a. Buat file source code xsh_freememory.c

Masuk ke xinu/shell, buat xsh_freememory.c



```
development-system [Running] - Oracle VM VirtualBox : 1
Activities gedit Wed 13 May, 00:17
xsh_freememory.c
~/xinu/shell

/*Nama : Candra Dinata */
/*Nim : 2311104061*/

#include <xinu.h>

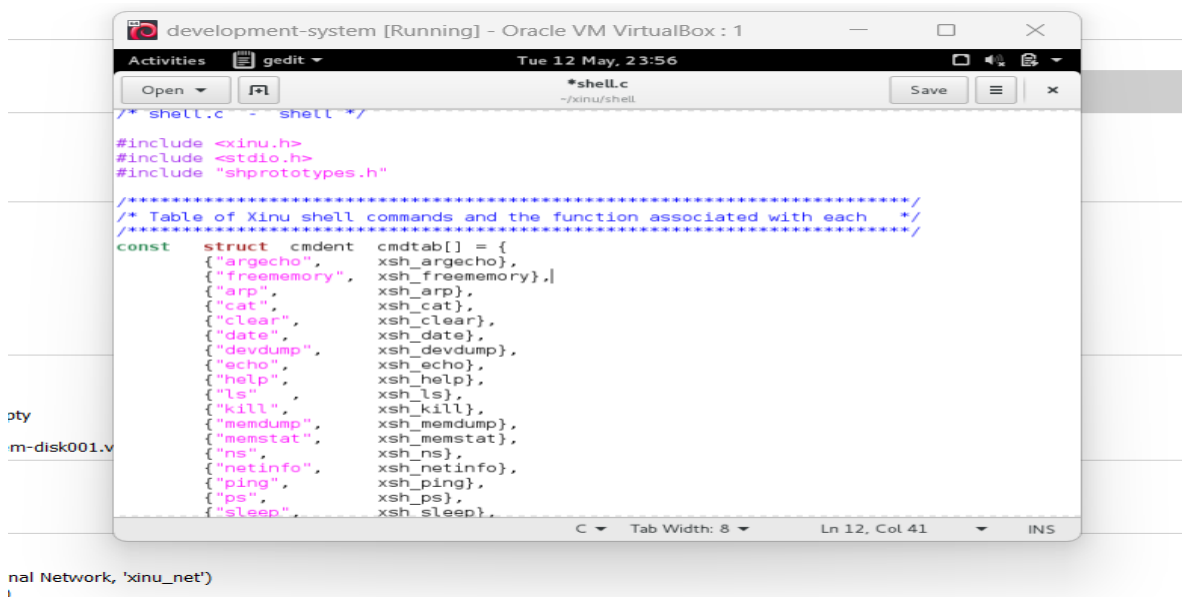
shellcmd xsh_freememory(int nargs, char *args[])
{
    struct memblk *memptr;
    uint32 totalFreeSpace = 0;

    printf("Free List:\n");
    printf("%-14s %-14s %-14s\n", "Block address", "Length (dec)", "Length (hex)");
    printf("%-14s %-14s %-14s\n", "-----", "-----", "-----");

    for (memptr = memlist.mnext; memptr != NULL; memptr = memptr->mnext){
        printf(" 0x%08x %10d 0x%08x\n",
            (uint32)memptr,
            (int32)memptr->length,
            memptr->length);
        totalFreeSpace += memptr->length;
    }

    printf("Total FreeSpace %10u\n", totalFreeSpace);
    return OK;
}
```

b. Daftarkan command di shell.c



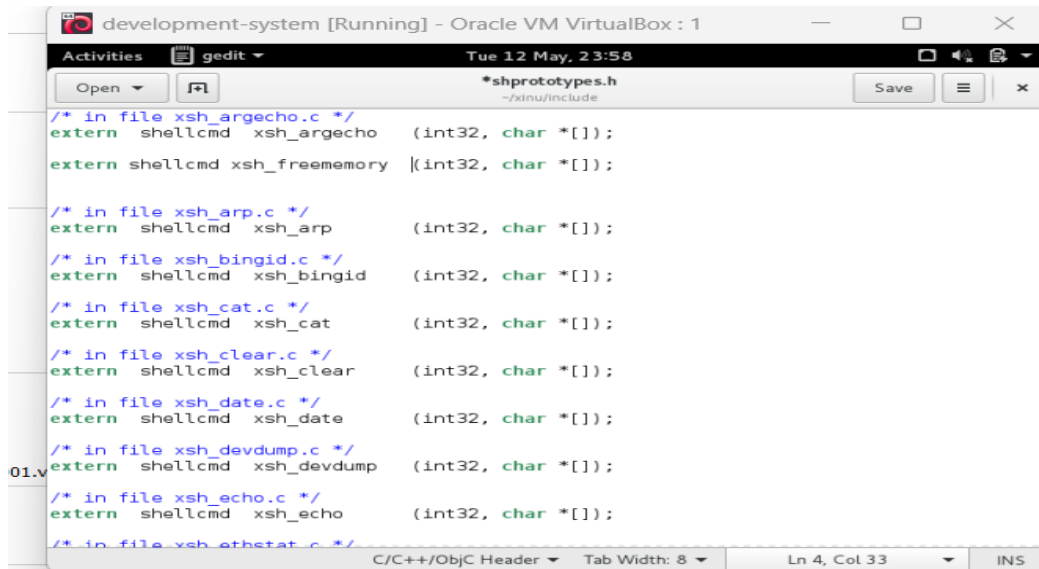
```
development-system [Running] - Oracle VM VirtualBox : 1
Activities gedit Tue 12 May, 23:56
shell.c
~/xinu/shell

/* shell.c - shell */

#include <xinu.h>
#include <stdio.h>
#include "shprototypes.h"

/* Table of Xinu shell commands and the function associated with each */
const struct cmdent cmdtab[] = {
    { "argecho", xsh_argecho },
    { "freememory", xsh_freememory },
    { "arp", xsh_arp },
    { "cat", xsh_cat },
    { "clear", xsh_clear },
    { "date", xsh_date },
    { "devdump", xsh_devdump },
    { "echo", xsh_echo },
    { "help", xsh_help },
    { "ls", xsh_ls },
    { "kill", xsh_kill },
    { "memdump", xsh_memdump },
    { "memstat", xsh_memstat },
    { "ns", xsh_ns },
    { "netinfo", xsh_netinfo },
    { "ping", xsh_ping },
    { "ps", xsh_ps },
    { "sleep", xsh_sleep },
}
```

c. Tambahkan deklarasi di shprototypes.h



```
development-system [Running] - Oracle VM VirtualBox : 1
Activities gedit Tue 12 May, 23:58
*shprototypes.h
~/xinu/include
Save
/* in file xsh_argecho.c */
extern shellcmd xsh_argecho (int32, char *[]);
extern shellcmd xsh_freememory (int32, char *[]);

/* in file xsh_arp.c */
extern shellcmd xsh_arp (int32, char *[]);

/* in file xsh_bingid.c */
extern shellcmd xsh_bingid (int32, char *[]);

/* in file xsh_cat.c */
extern shellcmd xsh_cat (int32, char *[]);

/* in file xsh_clear.c */
extern shellcmd xsh_clear (int32, char *[]);

/* in file xsh_date.c */
extern shellcmd xsh_date (int32, char *[]);

/* in file xsh_devdump.c */
extern shellcmd xsh_devdump (int32, char *[]);

/* in file xsh_echo.c */
extern shellcmd xsh_echo (int32, char *[]);

/* in file xsh_ethstat.c */
extern shellcmd xsh_ethstat (int32, char *[]);

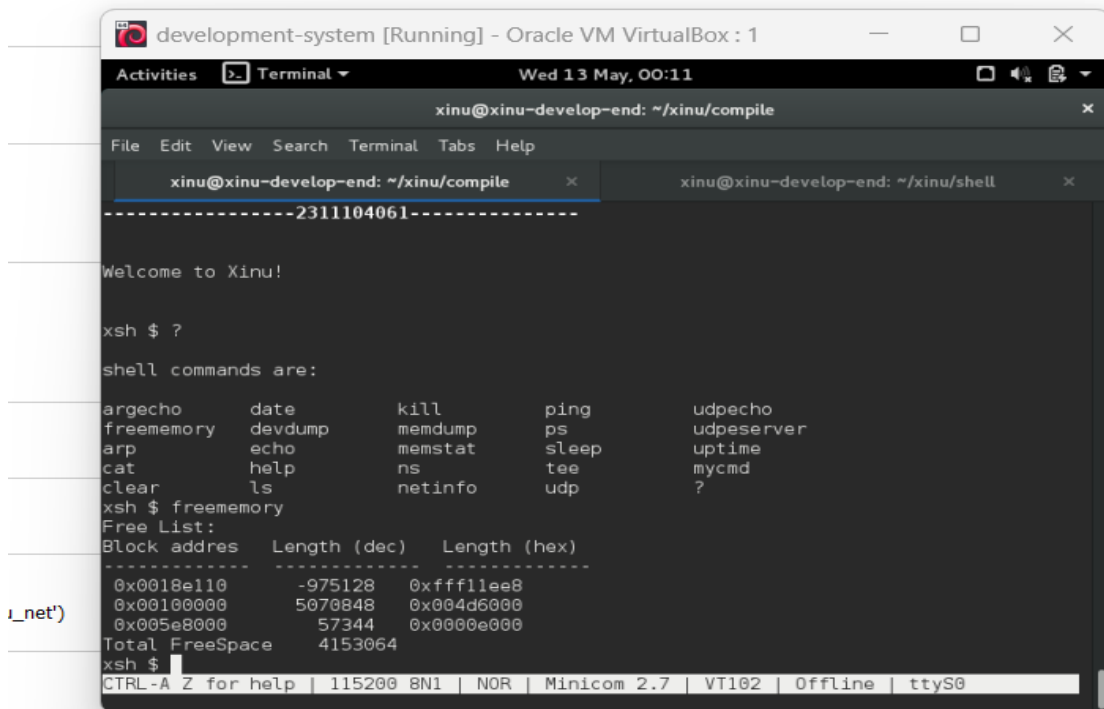
C/C++/ObjC Header Tab Width: 8 Ln 4, Col 33 INS
```

vork, 'xinu_net')

d. Compile

Masuk ke ~/xinu/compile, jalankan make lalu make

e. Testing output



```
development-system [Running] - Oracle VM VirtualBox : 1
Activities Terminal Wed 13 May, 00:11
xinu@xinu-develop-end: ~/xinu/compile
File Edit View Search Terminal Tabs Help
xinu@xinu-develop-end: ~/xinu/compile xinu@xinu-develop-end: ~/xinu/shell
-----2311104061-----
Welcome to Xinu!

xsh $ ?

shell commands are:

argecho      date        kill        ping        udpecho
freememory   devdump     memdump     ps          udpeserver
arp          echo        memstat     sleep       uptime
cat          help        ns          tee         mycmd
clear        ls          netinfo     udp         ?

xsh $ freememory
Free List:
Block address  Length (dec)  Length (hex)
-----
0x0018e110    -975128      0xffff11ee8
0x00100000     5070848      0x004d6000
0x005e8000     57344        0x0000e000
Total FreeSpace  4153064

xsh $
CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.7 | VT102 | Offline | ttyS0
```

i_net')

2. Jawaban Teori.

a. Mengapa Xinu memisahkan data segment dan BSS segment?

Xinu memisahkan *data segment* dan *BSS segment* untuk efisiensi penggunaan memori dan pengelolaan program.

- Data segment digunakan untuk variabel global atau static yang sudah memiliki nilai awal.

Contoh:

```
int angka = 10;
```

- BSS segment digunakan untuk variabel global atau static yang belum diinisialisasi.

Contoh:

```
int angka;
```

Pemisahan ini membuat ukuran file executable lebih kecil karena variabel pada BSS tidak perlu disimpan nilainya di file program. Saat program dijalankan, sistem hanya mengalokasikan memori dan mengisinya dengan nol secara otomatis.

b. Bagaimana alokasi dan dealokasi memori selama eksekusi memengaruhi ukuran free space?

Saat program melakukan alokasi memori (*malloc/getmem*), ukuran free space akan berkurang karena sebagian memori dipakai proses.

Sebaliknya, ketika memori dikembalikan (*free/freemem*), ukuran free space akan bertambah kembali.

Namun selama eksekusi, ukuran free space bisa menjadi tidak teratur akibat:

- proses alokasi berulang,
- ukuran memori berbeda-beda,
- dealokasi tidak berurutan.

Hal tersebut dapat menyebabkan fragmentasi memori sehingga walaupun total free space besar, memori belum tentu tersedia dalam blok besar yang berkesinambungan.

c. Mengapa penggunaan heap lebih berpotensi menimbulkan masalah dibandingkan stack?

Heap lebih berpotensi menimbulkan masalah karena pengelolaannya dilakukan secara dinamis oleh programmer atau sistem.

Masalah yang sering terjadi:

- Memory leak → memori lupa dibebaskan.
- Fragmentasi → banyak blok kecil terpisah.
- Double free → memori dibebaskan dua kali.
- Invalid access → mengakses memori yang sudah dibebaskan.

Sedangkan stack lebih aman karena:

- alokasi dan dealokasi berlangsung otomatis,
- menggunakan mekanisme LIFO (*Last In First Out*),
- lebih terstruktur dan cepat.

d. Mengapa Xinu menggunakan struktur linked list untuk menyimpan free block?

Xinu menggunakan linked list karena struktur ini fleksibel untuk mengelola blok memori yang ukurannya berubah-ubah secara dinamis.

Keuntungan linked list:

- mudah menambah atau menghapus free block,
- efisien untuk menggabungkan blok memori yang berdekatan,
- tidak memerlukan ukuran array tetap,
- cocok untuk sistem operasi sederhana seperti Xinu.

Setiap node linked list menyimpan:

- alamat blok memori,
- ukuran blok,
- pointer ke blok berikutnya.

e. Apa tantangan utama dalam penggunaan heap di Xinu?

Tantangan utama penggunaan heap di Xinu adalah pengelolaan memori dinamis agar tetap efisien dan tidak mengalami fragmentasi.

Beberapa tantangan utamanya:

1. Fragmentasi memori
Banyak blok kecil membuat memori sulit digunakan untuk alokasi besar.
2. Memory leak
Memori tidak dikembalikan setelah dipakai.
3. Kesalahan alokasi/dealokasi
Seperti double free atau invalid pointer.
4. Keterbatasan memori
Xinu biasanya berjalan pada sistem dengan resource terbatas.
5. Sinkronisasi akses memori
Jika banyak proses mengakses heap secara bersamaan, diperlukan mekanisme proteksi agar data heap tidak rusak.